DrivenData prize documentation — DaT Parkinson's Challenge

Final standing: **3rd place**. Answers below are numbered per the prize recipient solution documentation guide.

1. Who are you (mini-bio) and what do you do professionally?

I am a nuclear medicine physician. I have a triple background of medical formation, engineering sciences and business. I co-founded an AI-based company (PAIRE) five years ago, aiming to be an assistant for nuclear medicine physicians in their interpretation of PET/CT. My clinical practice is equally split between a university hospital (CHU Henri Mondor, APHP) and private clinical practice (CISEL Chartres, IMT Asterion).

Solo entry — no team.

2. What motivated you to compete in this challenge?

At first it was for fun — I wanted to dive more into AI-assisted coding and agents. Then I got challenged by other competitors and my ego took off.

3. High level summary of your approach: what did you do and why?

I quickly realized that 3D convolutional neural networks were largely worse than 2D ones, so I worked on projections and tried to maximize information related to priors such as posterior striatal denervation, asymmetry, and length-shortening of the striata.

Concretely: each 3D scan is reduced to a small stack of physically-meaningful 2D maps — four axial channels (peak uptake, mean uptake, anisotropy × uptake, and a normalized displacement-tail channel) plus per-hemisphere sagittal views — read by an ordinary 2D densenet121. This beat full-3D and striatal-crop-3D readers by a wide margin (0.043 and 0.035 log loss worse, respectively, across three seeds). The projection holds zero learnable parameters — an earlier learnable version collapsed to rank 1.7 of 5, so the fixed physical form is what remains. Striatal scale is deliberately never normalized: length alone reaches 0.89 AUC on its own, and normalizing it away would delete that signal. The final model is a 10-member × 5-fold ensemble, flip-TTA, calibrated by a single temperature slope.

4. Do you have any useful charts, graphs, or visualizations from the process?

Yes, in `docs/figures/` (referenced in the main README):

- `preprocessing.png` — the five preprocessing stages on one abnormal scan, from raw nifti to the canonicalized, ellipse-localized box.
- `inputs.png` — the twelve input channels the network actually receives, for a confident normal, a confident abnormal, and a mild (borderline) abnormal scan side by side — the mild case is where most of the loss concentrates.

- `lesion.png` — one real normal scan, progressively synthetically denervated, with the winning model's own prediction on each step.
- `channels.png`, `discordant.png` — channel-level and discordant-case visualizations used during development.

5. Copy and paste the 3 most impactful parts of your code and explain what each does and how it helped your model.

Canonization (`inference/canonize.py`) — an 8-DOF warp (translation + yaw; scale deliberately excluded) that puts every scan's striatal ellipsoid into a shared reference frame before projection, using a per-hemisphere ellipse localiser. This is what lets a zero-parameter projection work at all: without a consistent anatomical frame, the fixed physical channels below would be comparing different anatomy scan to scan.

DatProj — `PhysShape3N` (`datscan/projection.py`) — reduces the superior-inferior axis to four fixed channels (peak uptake, mean uptake, anisotropy × uptake, normalized displacement-tail), with **zero learnable parameters**. It is the core representational choice of the whole approach: a learnable version of the same idea was tried and collapsed to rank 1.7 of 5, so the fixed, physically-motivated form is what survives. `tau = 4.090·mu + 1.259` is the one live scalar, set per scan from its own mean, which is what gives it robustness across the ten centres' acquisition differences.

Axial-sagittal fusion (`sagfuse=attnres`, in `datscan/model.py`) — stacks the left and right sagittal projections as eight channels through one shared stem, so the two hemispheres meet at the very first convolution, then fuses into the axial trunk through a zero-initialized attention gate (the model starts as pure-axial and learns the correction). Deferring the hemisphere comparison to any later layer measurably lost (a mid-level Siamese variant cost 0.0056 log loss); comparing at conv-1 is where the asymmetry signal lives, and asymmetry is one of the three priors (asymmetry, posterior-first denervation, striatal shortening) the whole design targets.

6. What features or areas of the image were most important to your performance?

The striatum, read through three specific priors rather than the raw volume:

- **Asymmetry between hemispheres** — carried by the sagittal fusion at conv-1 (see #5).
- **Posterior-to-anterior denervation gradient** — early Parkinson's loses posterior putamen first, spreading anteriorly and to the caudate later; this ordering is encoded directly in the antero-posterior projection channels and in how synthetic lesions are generated for augmentation.
- **Striatal length / shape shortening** — striatal length alone reaches 0.89 AUC; scale is deliberately never normalized away, since doing so would delete this signal.

The mild/borderline-denervation band is where the axial channels of an abnormal scan look closest to a normal one, and is where nearly all of the remaining loss concentrates.

7. Please provide the machine specs and time you used to run your model.

- **CPU (model):** 24-core (unspecified model), 12 preprocessing workers used at inference
- **GPU (model or N/A):** 3× Tesla V100S-PCIE-32GB (training); any single ≥11 GB CUDA GPU
  suffices for inference
- **Memory (GB):** 252 GB available for training; 16 GB is enough for inference
- **OS:** Ubuntu 22.04
- **Train duration:** ~2 h 15 per fold × 50 folds (10 members × 5 folds) ≈ 12 h wall clock
  on 3 GPUs in parallel
- **Inference duration:** well inside the competition's 3 h / ~3000-scan budget (measured
  at roughly 3.4 s/scan on the verification run); the shipped `inference/main.py` runs
  serially preprocessed in a process pool one batch ahead of the GPU

8. Anything we should watch out for or be aware of in using your model?

No.

9. Did you use any tools for data preparation or exploratory data analysis that aren't listed in your code submission?

No. I used Claude Code as an assistant throughout — for idea exploration, launching training runs, creating agents with specific rules for code writing, review and testing, model evaluation, and optimizing/planning the submission (all programmed in advance).

10. How did you evaluate performance of the model other than the provided metric, if at all?

I struggled to find a good local proxy for leaderboard performance, so I came up with strict rules for whether a change counts as an improvement — it had to hold up across:

- **3 seeds × 3 folds** (a screen below three seeds is not trustworthy — per-fold seed
  noise alone runs 0.005–0.009 log loss for densenet and 0.02–0.07 for efficientnet-b0)
- **Leave-one-center-out** — generalization across the ten acquisition centres/scanners
- **LOMO** — leave-one-mild-out, using a severity index that buckets scans into three
  clusters by degree of denervation, to make sure gains weren't concentrated in the easy
  (clearly normal/clearly abnormal) cases at the expense of the mild band

11. What are some other things you tried that didn't necessarily make it into the final workflow (quick overview)?

Far too much to summarize here — the full list with each one's measured cost is in `notes/REFUTED.md`. Briefly, refuted directions included: fully-3D and striatal-crop-3D readers, a learnable variant of the projection layer, several augmentation removals that looked like in-distribution wins but lost on held-out-cluster validation, alternative lesion-synthesis variants (rim-first thinning, pre-transform lesioning, a diversified synthetic-field variant), and a mid-level Siamese fusion instead of conv-1 fusion.

12. If you were to continue working on this problem for the next year, what methods or techniques might you try in order to build on your work so far? Are there other fields or features you felt would have been very helpful to have?

I would add clinical data — age and sex may alter normal striatal length, and the model currently has no way to condition on that. I would also explore why some architectures that worked really well on the training set didn't transfer well, notably equivariant CNNs.

13. What simplifications could be made to run your solution faster without sacrificing significant accuracy?

It's already really fast.